Air University

Data Structures & Algorithms (Lab)

Year 2025

Mr. Ali Raza

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: December 19, 2025

Project Report

Title: - Password Vault System

1. Introduction

As people and organizations navigate the digital world, managing multiple online accounts has become essential. Keeping passwords secure is crucial because manual storage is prone to errors and cyber threats. This project introduces a Password Vault Management System. It allows users to store, retrieve, and manage passwords securely while applying advanced Data Structures and Algorithms (DSA) concepts. The system focuses on efficient storage, retrieval, and updates. It also emphasizes performance and security and uses programming techniques like file saving and encryption. The implementation occurs in C++, combining Object-Oriented Programming (OOP) with key DSA concepts.

2. Objectives

- Create a secure system for storing login details (site, username, password).
- Use a balanced AVL Tree to ensure fast search, addition, and deletion.
- Implement a stack to provide an undo feature for the last actions.
- Use a queue to track login attempts and help prevent brute-force attacks.
- Ensure data is saved by storing encrypted passwords in files.
- Protect passwords with masking and an option to reveal them.

3. Key Features

i. **Add Entry**
Users can input site credentials while checking for duplicates and choosing to overwrite existing information. Passwords are encrypted using a Caesar Cipher and entered the AVL tree.

ii. **Delete Entry**
Users can remove site credentials, storing deleted entries in a stack for undo functionality.

iii. **View Entry**
Users can selectively view credentials with masked passwords by default and a temporary reveal option for decrypted passwords.

iv. **Undo Operation**
A stack records recent actions, enabling users to reverse the last add or delete operation for data integrity.

v. **View All Sites**
Users can see all stored credentials in alphabetical order by site name by traversing the AVL tree.

vi. **Action Logging**
The system logs all actions (add, delete, undo) with timestamps to maintain a record of user activity.

vii. **Persistence**
Encrypted credentials are safely stored in Vault.dat, while action logs are recorded in Actions.txt.

4. Data Structures Used

i. **AVL Tree:** This structure stores all credentials using the site name as the key, ensuring quick insertion, deletion, and search with O (log n) complexity. Self-balancing rotations include LL (Left-Left), RR (Right-Right), LR (Left-Right), and RL (Right-Left).

ii. **Stack:** It keeps a record of add/delete operations for the undo feature, following the Last-In-First-Out principle.

iii. **Queue:** This structure tracks login attempts to help prevent brute-force attacks, using the First-In-First-Out principle.

iv. **File Storage:** Encrypted passwords are saved in Vault.dat using Hex encoding, while action logs are stored in Actions.txt with timestamps.

5. Algorithms and Techniques

i. **AVL Tree Operations**

- **Insertion:** Add new credentials to the tree, with automatic rebalancing if needed.
- **Deletion:** Remove nodes, using successor replacement for nodes with two children.
- **Search:** Retrieve credentials quickly in O (log n).

ii. **Password Security:**

- **Encryption/Decryption:** Encrypt passwords with a Caesar cipher before saving.
- **Masked Display:** Passwords are hidden by default, with an option to show them temporarily.

iii. **Undo Implementation:**

- The stack logs operations like "ADD site" or "DELETE site,username,password," allowing users to reverse the last action and update both the tree and files.

iv. **File Persistence:**

- The AVL tree saves data immediately after changes, keeping encrypted credentials in Vault.dat for security.

6. Program Workflow

i. **Authentication:**
Users need a master password, stored as a simple hash, to access the vault, with a queue to limit repeated failed login attempts.

ii. **Menu-driven System:**
This system offers options to add, delete, view entries, undo actions, view all sites, check action logs, and save/exit.

iii. **Operational Flow:**
Users select an action, the system processes it, updates the AVL tree, logs the action, and saves to the file. The undo stack helps safely revert the last action.

7. Implementation Details

Programming Language: C++

i. **Concepts Used:**
   a. Object-Oriented Programming (Classes: AVLTree, Stack, Queue, Cipher, FileManager, PasswordVault)
   b. Dynamic Memory Management
   c. File Input/Output and Data Storage
   d. Time-stamped Logging

ii. **Security Measures:**
   a. Passwords are encrypted and displayed in a masked format.
   b. The system verifies the master password and limits attempts to increase security.

iii. **Efficiency:**
   a. The AVL tree stays balanced, which keeps search and insertion times short.
   b. Operations with the Stack and Queue run in constant time.

8. Observations & Learning Outcomes

- I learned how to use AVL trees in real situations.
- I integrated several data structures in one project.
- I gained experience in file handling with encryption for secure data storage.
- I created an undo feature, showing how stacks can be useful in systems.
- I learned secure coding practices and how to design modular projects.

9. Conclusion

The Password Vault Management System shows how data structures and algorithms can be applied practically. It uses AVL trees, stack, and queue functions, along with file storage and encryption, to create a secure and user-friendly system. This project meets the DSA curriculum requirements and highlights the real-world use of data structures, making it suitable for a third-semester project presentation.

10. Future Enhancements

- Use stronger password hashing (SHA-256) for the master password.
- Mask password input while typing (using getch() or terminal control).
- Add a graphical user interface to improve user experience.
- Support multiple users with role-based access controls.

11. Code

```
#include <iostream>

#include <string>
#include <sstream>
#include <fstream>
#include <limits>
#include <ctime>
#include <algorithm>
#include <iomanip>
#include <regex>
#include <functional>
#include <vector>
using namespace std;

class stackNode // Node for stack implementation
{
public:
    string data;
    stackNode *next = nullptr;
};

class Stack // Stack class for undo operations
{
private:
    stackNode *top;
    int currentSize;

public:
    Stack() : top(nullptr), currentSize(0) {}
```

```
    ~Stack()
    {
        while (!isEmpty())
            pop();
    }
    void push(string value) // Push value onto stack if not empty
    {
        if (value.empty())
            return;
        stackNode *newNode = new stackNode();
        newNode->data = value;
        newNode->next = top;
        top = newNode;
        currentSize++;
    }
    string pop() // Pop and return top value
    {
        if (isEmpty())
            return "";
        stackNode *temp = top;
        top = top->next;
        string value = temp->data;
        delete temp;
        currentSize--;
        return value;
    }
    bool isEmpty() { return top == nullptr; }
};

class queueNode // Node for queue implementation
{
public:
    string data;
    queueNode *next = nullptr;
};

class Queue // Queue class for rate-limiting authentication attempts
{
private:
    queueNode *front, *rear;
    int currentSize;

public:
    Queue() : front(nullptr), rear(nullptr), currentSize(0) {}
    ~Queue() { clearQueue(); }
    void enqueue(string item) // Add item to queue if not empty
    {
        if (item.empty())
            return;
        queueNode *newNode = new queueNode();
        newNode->data = item;
        newNode->next = nullptr;
```

```
        if (isEmpty())
            front = rear = newNode;
        else
        {
            rear->next = newNode;
            rear = newNode;
        }
        currentSize++;
    }
    string dequeue() // Remove and return front item
    {
        if (isEmpty())
            return "";
        queueNode *temp = front;
        front = front->next;
        if (!front)
            rear = nullptr;
        string value = temp->data;
        delete temp;
        currentSize--;
        return value;
    }
    string peek() { return isEmpty() ? "" : front->data; }
    int size() { return currentSize; }
    bool isEmpty() { return front == nullptr; }
    void clearQueue() // Clear all items in queue
    {
        while (!isEmpty())
            dequeue();
    }
};

class AuthSystem // Class for handling authentication and master password management
{
private:
    string MasterHash;
    Queue attempts;

public:
    void LoadMasterPassword() // Load master password hash from file
    {
        ifstream file("Master.dat");
        if (file.is_open())
        {
            getline(file, MasterHash);
            file.close();
        }
        else
            MasterHash = "";
    }

    void saveMasterPassword() // Save master password hash to file
```

```
{
    ofstream file("Master.dat");
    if (file.is_open())
    {
        file << MasterHash << endl;
        file.close();
    }
    else
        cout << "Failed to save master password." << endl;
}

string simpleHash(string text) // Simple hash function for password hashing
{
    long long hashValue = 0;
    for (char c : text)
        hashValue = (hashValue * 31 + c) % 10000000;
    return to_string(hashValue);
}

void setMasterPassword() // Set master password by prompting user
{
    string pass;
    cout << "Set Master Password: ";
    getline(cin, pass);
    MasterHash = simpleHash(pass);
    saveMasterPassword();
    cout << "Master Password Set.\n";
}

bool authenticate() // Authenticate user with master password, rate-limited
{
    while (true)
    {
        string inputPass;
        cout << "Enter Master Password: ";
        getline(cin, inputPass);
        if (simpleHash(inputPass) == MasterHash)
        {
            attempts.clearQueue();
            return true;
        }
        else
        {
            attempts.enqueue(to_string(time(nullptr)));
            cout << "Incorrect Password.\n";
            if (attempts.size() > 5)
            {
                cout << "Too Many Attempts! Try Later.\n";
                return false;
            }
        }
    }
```

```
    }
    string getMasterHash() { return MasterHash; }
};

string toHex(const string &s) // Convert string to hexadecimal representation
{
    ostringstream oss;
    for (unsigned char c : s)
        oss << hex << setw(2) << setfill('0') << (int)c;
    return oss.str();
}

string fromHex(const string &hex) // Convert hexadecimal string back to original string
{
    string out;
    out.reserve(hex.size() / 2);
    for (size_t i = 0; i < hex.size(); i += 2)
    {
        string byte = hex.substr(i, 2);
        char chr = (char)strtol(byte.c_str(), nullptr, 16);
        out.push_back(chr);
    }
    return out;
}

// Validation helper functions
bool isStrongPassword(const string &password)
{
    if (password.length() < 8 || password.length() > 100)
        return false;
    bool hasUpper = false, hasLower = false, hasDigit = false, hasSpecial = false;
    for (char c : password)
    {
        if (isupper(c))
            hasUpper = true;
        else if (islower(c))
            hasLower = true;
        else if (isdigit(c))
            hasDigit = true;
        else if (!isspace(c))
            hasSpecial = true;
    }
    return hasUpper && hasLower && hasDigit && hasSpecial;
}

bool isValidUsername(const string &username)
{
    if (username.length() < 3 || username.length() > 20)
        return false;
    regex pattern("^[a-zA-Z0-9_]+$");
    return regex_match(username, pattern);
}
```

```
bool isValidSitename(const string &sitename)
{
    if (sitename.empty() || sitename.length() > 100 || sitename.find(' ') != string::npos)
        return false;
    regex pattern("^[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}$");
    return regex_match(sitename, pattern);
}
class Cipher // Simple Caesar cipher for password encryption/decryption
{
private:
    int key = 3;

public:
    string encryption(string text) // Encrypt text by shifting characters
    {
        string encrypted = "";
        for (char c : text)
            encrypted += (c + key);
        return encrypted;
    }
    string decryption(string text) // Decrypt text by shifting characters back
    {
        string decrypted = "";
        for (char c : text)
            decrypted += (c - key);
        return decrypted;
    }
};

class treeNode // Node for AVL tree storing credentials
{
public:
    string key, username, password;
    int height;
    treeNode *left = nullptr, *right = nullptr;
};

class AVLTree // AVL tree class for storing and managing password credentials
{
private:
    treeNode *root = nullptr;

    treeNode *createNode(string key, string username, string password) // Create a new
tree node
    {
        treeNode *n = new treeNode();
        n->key = key;
        n->username = username;
        n->password = password;
        n->height = 1;
        return n;
```

```
    }

    int getHeight(treeNode *n) { return n ? n->height : 0; }
    int getBalance(treeNode *n) { return n ? getHeight(n->left) - getHeight(n->right) : 0;
}

    treeNode *LL_Rotation(treeNode *l1) // Left-Left rotation for balancing
    {
        treeNode *l2 = l1->left;
        treeNode *l3 = l2->right;
        l2->right = l1;
        l1->left = l3;
        l1->height = max(getHeight(l1->left), getHeight(l1->right)) + 1;
        l2->height = max(getHeight(l2->left), getHeight(l2->right)) + 1;
        return l2;
    }

    treeNode *RR_Rotation(treeNode *r1) // Right-Right rotation for balancing
    {
        treeNode *r2 = r1->right;
        treeNode *r3 = r2->left;
        r2->left = r1;
        r1->right = r3;
        r1->height = max(getHeight(r1->left), getHeight(r1->right)) + 1;
        r2->height = max(getHeight(r2->left), getHeight(r2->right)) + 1;
        return r2;
    }

    treeNode *LR_Rotation(treeNode *lr1) // Left-Right rotation for balancing
    {
        lr1->left = RR_Rotation(lr1->left);
        return LL_Rotation(lr1);
    }

    treeNode *RL_Rotation(treeNode *rl1) // Right-Left rotation for balancing
    {
        rl1->right = LL_Rotation(rl1->right);
        return RR_Rotation(rl1);
    }

    treeNode *insertNode(treeNode *node, string key, string username, string password) //
Insert node into AVL tree with balancing
    {
        if (!node)
            return createNode(key, username, password);
        else if (key < node->key)
            node->left = insertNode(node->left, key, username, password);
        else if (key > node->key)
            node->right = insertNode(node->right, key, username, password);
        else
        {
            node->username = username;
```

```
            node->password = password;
            return node;
        }

        node->height = 1 + max(getHeight(node->left), getHeight(node->right));
        int bal = getBalance(node);

        if (bal > 1 && key < node->left->key)
            return LL_Rotation(node);
        if (bal < -1 && key > node->right->key)
            return RR_Rotation(node);
        if (bal > 1 && key > node->left->key)
            return LR_Rotation(node);
        if (bal < -1 && key < node->right->key)
            return RL_Rotation(node);

        return node;
    }

    treeNode *searchNode(treeNode *node, string key) // Search for a node by key
    {
        if (!node)
            return nullptr;
        if (key == node->key)
            return node;
        else if (key < node->key)
            return searchNode(node->left, key);
        else
            return searchNode(node->right, key);
    }

    treeNode *findSuccessor(treeNode *node) // Find inorder successor
    {
        while (node && node->left)
            node = node->left;
        return node;
    }

    treeNode *deleteNode(treeNode *node, string key) // Delete node from AVL tree with
balancing
    {
        if (!node)
            return nullptr;
        if (key < node->key)
            node->left = deleteNode(node->left, key);
        else if (key > node->key)
            node->right = deleteNode(node->right, key);
        else
        {
            if (!node->left)
            {
                treeNode *temp = node->right;
```

```
                delete node;
                return temp;
            }
            else if (!node->right)
            {
                treeNode *temp = node->left;
                delete node;
                return temp;
            }
            else
            {
                treeNode *succ = findSuccessor(node->right);
                node->key = succ->key;
                node->username = succ->username;
                node->password = succ->password;
                node->right = deleteNode(node->right, succ->key);
            }
        }

        node->height = 1 + max(getHeight(node->left), getHeight(node->right));
        int bal = getBalance(node);
        if (bal > 1 && getBalance(node->left) >= 0)
            return LL_Rotation(node);
        if (bal > 1 && getBalance(node->left) < 0)
            return LR_Rotation(node);
        if (bal < -1 && getBalance(node->right) <= 0)
            return RR_Rotation(node);
        if (bal < -1 && getBalance(node->right) > 0)
            return RL_Rotation(node);
        return node;
    }

    void InOrder(treeNode *node) // Inorder traversal to display sites and usernames
    {
        if (!node)
            return;
        InOrder(node->left);
        cout << "Site: " << node->key << " | Username: " << node->username << endl;
        InOrder(node->right);
    }

public:
    treeNode *getRoot() { return root; }
    void setRoot(treeNode *r) { root = r; }
    treeNode *insertCredentials(treeNode *node, string key, string username, string
password)
    {
        return insertNode(node, key, username, password);
    }
    treeNode *searchCredentials(treeNode *node, string key) { return searchNode(node,
key); }
    treeNode *deleteCredentials(string key)
```

```
    {
        root = deleteNode(root, key);
        return root;
    }
    void traverseInorder() { InOrder(root); }
};

class FileManager // Class for handling file operations for saving/loading vault data and
logging actions
{
public:
    void saveTree(treeNode *node, ofstream &file) // Recursively save tree nodes to file
    {
        if (!node)
            return;
        file << node->key << ' ' << node->username << ' ' << toHex(node->password) <<
'\n';
        saveTree(node->left, file);
        saveTree(node->right, file);
    }

    void saveAll(AVLTree *tree) // Save entire vault tree to file
    {
        ofstream file("Vault.dat");
        if (file.is_open())
        {
            saveTree(tree->getRoot(), file);
            file.close();
        }
        else
            cout << "Error saving Vault.dat\n";
    }

    void loadAll(AVLTree *tree) // Load vault data from file into tree
    {
        ifstream file("Vault.dat");
        if (!file.is_open())
            return;
        string line;
        while (getline(file, line))
        {
            if (line.empty())
                continue;
            stringstream ss(line);
            string key, username, passwordHex;
            if (!(ss >> key >> username >> passwordHex))
                continue;
            string password = fromHex(passwordHex);
            tree->setRoot(tree->insertCredentials(tree->getRoot(), key, username,
password));
        }
        file.close();
```

```
    }

    void logAction(string action) // Log an action with timestamp to file
    {
        time_t now = time(nullptr);
        char buffer[80];
        strftime(buffer, sizeof(buffer), "%Y-%m-%d %H:%M:%S", localtime(&now));
        ofstream file("Actions.txt", ios::app);
        if (file.is_open())
        {
            file << "[" << buffer << "] " << action << '\n';
            file.close();
        }
    }

    void saveSites(treeNode *node, ofstream &file) // Helper to save sites and usernames
recursively
    {
        if (!node)
            return;
        saveSites(node->left, file);
        file << "Site: " << node->key << " | Username: " << node->username << endl;
        saveSites(node->right, file);
    }

    void saveAllSitesAndUsernames(AVLTree *tree) // Save all sites and usernames to file
    {
        ofstream file("AllSites.txt");
        if (file.is_open())
        {
            saveSites(tree->getRoot(), file);
            file.close();
        }
        else
            cout << "Error saving AllSites.txt\n";
    }
};

class PasswordVault // Main class for managing password vault operations
{
private:
    AVLTree tree;
    Stack undo;
    Cipher cipher;
    FileManager &logger;

public:
    PasswordVault(FileManager &fm) : logger(fm) {}

    void addEntry() // Add a new password entry
    {
        try
```

```
        {
            string site, username, password;
            do
            {
                cout << "Enter Site: ";
                getline(cin, site);
                if (!isValidSitename(site))
                {
                    cout << "Sitename must be a valid domain name (e.g., example.com).\n";
                }
            } while (!isValidSitename(site));

            treeNode *existing = tree.searchCredentials(tree.getRoot(), site);
            if (existing)
            {
                char choice;
                cout << "Site exists. Overwrite? (y/n): ";
                cin >> choice;
                cin.ignore(numeric_limits<streamsize>::max(), '\n');
                if (choice != 'y' && choice != 'Y')
                {
                    cout << "Add cancelled.\n";
                    return;
                }
            }

            do
            {
                cout << "Enter Username: ";
                getline(cin, username);
                if (!isValidUsername(username))
                {
                    cout << "Username must be 3-20 characters long and contain only
letters, numbers, and underscores.\n";
                }
            } while (!isValidUsername(username));

            do
            {
                cout << "Enter Password: ";
                getline(cin, password);
                if (!isStrongPassword(password))
                {
                    cout << "Password must be at least 8 characters long and include
uppercase, lowercase, digits, and special characters.\n";
                }
            } while (!isStrongPassword(password));

            string encrypted = cipher.encryption(password);
            tree.setRoot(tree.insertCredentials(tree.getRoot(), site, username,
encrypted));
```

```
            undo.push("ADD " + site);
            logger.logAction("Added site: " + site);
            logger.saveAll(&tree);
            logger.saveAllSitesAndUsernames(&tree);
            cout << "Entry added.\n";
        }
        catch (exception &e)
        {
            cout << "Error: " << e.what() << endl;
        }
    }

    void deleteEntry() // Delete an existing password entry
    {
        try
        {
            int deleteChoice;
            string deleteChoiceStr;
            do
            {
                cout << "Delete by (1) Site or (2) Username: ";
                getline(cin, deleteChoiceStr);
                stringstream ssChoice(deleteChoiceStr);
                if (!(ssChoice >> deleteChoice) || (deleteChoice != 1 && deleteChoice !=
2))
                {
                    cout << "Invalid choice. Please enter 1 or 2.\n";
                }
            } while (deleteChoice != 1 && deleteChoice != 2);

            if (deleteChoice == 1)
            {
                // Delete by site
                string site;
                do
                {
                    cout << "Enter Site: ";
                    getline(cin, site);
                    if (!isValidSitename(site))
                    {
                        cout << "Sitename must be a valid domain name (e.g.,
example.com).\n";
                    }
                } while (!isValidSitename(site));

                treeNode *node = tree.searchCredentials(tree.getRoot(), site);
                if (!node)
                {
                    cout << "Nothing to delete.\n";
                    return;
                }
```

```
                string username = node->username;
                string backup = "DELETE " + site + "," + username + "," + node->password;
// Store encrypted password in undo stack
                undo.push(backup);

                tree.deleteCredentials(site);

                logger.logAction("Deleted site: " + site);
                logger.saveAll(&tree);
                logger.saveAllSitesAndUsernames(&tree);
                cout << "Deleted site: " << site << " (Username: " << username << ")\n";

                // Save deleted entry to file
                ofstream deletedFile("DeletedEntries.txt", ios::app);
                if (deletedFile.is_open())
                {
                    deletedFile << "Site: " << site << " | Username: " << username <<
endl;
                    deletedFile.close();
                }
                else
                {
                    cout << "Warning: Could not save to DeletedEntries.txt\n";
                }
            }
            else
            {
                // Delete by username
                cout << "Enter Username: ";
                string username;
                getline(cin, username);

                // Traverse tree to find all entries with matching username
                vector<pair<string, string>> matches;
                function<void(treeNode *)> findByUsername = [&](treeNode *node)
                {
                    if (!node)
                        return;
                    findByUsername(node->left);
                    if (node->username == username)
                    {
                        matches.push_back({node->key, node->username});
                    }
                    findByUsername(node->right);
                };
                findByUsername(tree.getRoot());

                if (matches.empty())
                {
                    cout << "No entries found for username: " << username << endl;
                    return;
                }
```

```
                cout << "Matching entries:\n";
                for (size_t i = 0; i < matches.size(); ++i)
                {
                    cout << i + 1 << ". Site: " << matches[i].first << " | Username: " <<
matches[i].second << endl;
                }

                int selection;
                string selectionStr;
                do
                {
                    cout << "Enter the number of the entry to delete: ";
                    getline(cin, selectionStr);
                    stringstream ss(selectionStr);
                    if (!(ss >> selection) || selection < 1 || selection >
(int)matches.size())
                    {
                        cout << "Invalid selection. Please enter a number between 1 and "
<< matches.size() << ".\n";
                    }
                } while (selection < 1 || selection > (int)matches.size());

                string site = matches[selection - 1].first;
                treeNode *node = tree.searchCredentials(tree.getRoot(), site);
                if (!node)
                {
                    cout << "Entry not found.\n";
                    return;
                }

                string backup = "DELETE " + site + "," + node->username + "," + node-
>password; // Store encrypted password in undo stack
                undo.push(backup);

                tree.deleteCredentials(site);

                logger.logAction("Deleted site: " + site);
                logger.saveAll(&tree);
                logger.saveAllSitesAndUsernames(&tree);
                cout << "Deleted site: " << site << " (Username: " << node->username <<
")\n";

                // Save deleted entry to file
                ofstream deletedFile("DeletedEntries.txt", ios::app);
                if (deletedFile.is_open())
                {
                    deletedFile << "Site: " << site << " | Username: " << node->username
<< endl;
                    deletedFile.close();
                }
                else
```

```
                {
                    cout << "Warning: Could not save to DeletedEntries.txt\n";
                }
            }
        }
        catch (exception &e)
        {
            cout << "Error: " << e.what() << endl;
        }
    }

    void viewEntry() // View a specific password entry
    {
        try
        {
            int searchChoice;
            string searchChoiceStr;
            do
            {
                cout << "Search by (1) Site or (2) Username: ";
                getline(cin, searchChoiceStr);
                stringstream ssChoice(searchChoiceStr);
                if (!(ssChoice >> searchChoice) || (searchChoice != 1 && searchChoice !=
2))
                {
                    cout << "Invalid choice. Please enter 1 or 2.\n";
                }
            } while (searchChoice != 1 && searchChoice != 2);

            if (searchChoice == 1)
            {
                // Search by site
                string site;
                do
                {
                    cout << "Enter Site: ";
                    getline(cin, site);
                    if (!isValidSitename(site))
                    {
                        cout << "Sitename must be a valid domain name (e.g.,
example.com).\n";
                    }
                } while (!isValidSitename(site));

                treeNode *node = tree.searchCredentials(tree.getRoot(), site);
                if (!node)
                {
                    cout << "No such site.\n";
                    return;
                }

                string decrypted = cipher.decryption(node->password);
```

```
                cout << "Site: " << site << "\nUsername: " << node->username <<
"\nPassword: ";

                for (size_t i = 0; i < decrypted.length(); i++)
                    cout << '*';
                cout << endl;

                char reveal;
                do
                {
                    cout << "Reveal password? (y/n): ";
                    cin >> reveal;
                    cin.ignore(numeric_limits<streamsize>::max(), '\n');
                } while (reveal != 'y' && reveal != 'Y' && reveal != 'n' && reveal !=
'N');
                if (reveal == 'y' || reveal == 'Y')
                    cout << "Password: " << decrypted << endl;
            }
            else
            {
                // Search by username
                cout << "Enter Username: ";
                string username;
                getline(cin, username);

                // Traverse tree to find all entries with matching username
                bool found = false;
                function<void(treeNode *)> searchByUsername = [&](treeNode *node)
                {
                    if (!node)
                        return;
                    searchByUsername(node->left);
                    if (node->username == username)
                    {
                        found = true;
                        string decrypted = cipher.decryption(node->password);
                        cout << "Site: " << node->key << "\nUsername: " << node->username
<< "\nPassword: ";
                        for (size_t i = 0; i < decrypted.length(); i++)
                            cout << '*';
                        cout << endl;

                        char reveal;
                        do
                        {
                            cout << "Reveal password for " << node->key << "? (y/n): ";
                            cin >> reveal;
                            cin.ignore(numeric_limits<streamsize>::max(), '\n');
                        } while (reveal != 'y' && reveal != 'Y' && reveal != 'n' && reveal
!= 'N');
                        if (reveal == 'y' || reveal == 'Y')
                            cout << "Password: " << decrypted << endl;
```

```
                        cout << endl;
                    }
                    searchByUsername(node->right);
                };
                searchByUsername(tree.getRoot());
                if (!found)
                {
                    cout << "No entries found for username: " << username << endl;
                }
            }
        }
        catch (exception &e)
        {
            cout << "Error: " << e.what() << endl;
        }
    }

    void undoAction() // Undo the last action
    {
        try
        {
            if (undo.isEmpty())
            {
                cout << "Nothing to undo.\n";
                return;
            }

            string actionString = undo.pop();
            size_t pos = actionString.find(" ");
            string type = actionString.substr(0, pos);
            string data = actionString.substr(pos + 1);

            if (type == "ADD")
            {
                cout << "Undoing addition of site: " << data << endl;
                tree.deleteCredentials(data);
                logger.logAction("Undo add site: " + data);
            }
            else if (type == "DELETE")
            {
                size_t c1 = data.find(","), c2 = data.find(",", c1 + 1);
                string site = data.substr(0, c1);
                string username = data.substr(c1 + 1, c2 - c1 - 1);
                string password = data.substr(c2 + 1);

                cout << "Undoing deletion of site: " << site << " (Username: " << username
<< ")" << endl;
                tree.setRoot(tree.insertCredentials(tree.getRoot(), site, username,
password)); // Password already encrypted
                logger.logAction("Undo delete site: " + site);

                // Remove from DeletedEntries.txt
```

```
            ifstream inFile("DeletedEntries.txt");
            ofstream outFile("temp.txt");
            string line;
            string target = "Site: " + site + " | Username: " + username;
            while (getline(inFile, line))
            {
                if (line != target)
                {
                    outFile << line << endl;
                }
            }
            inFile.close();
            outFile.close();
            remove("DeletedEntries.txt");
            rename("temp.txt", "DeletedEntries.txt");
        }

        logger.saveAll(&tree);
        logger.saveAllSitesAndUsernames(&tree);
        cout << "Undo operation completed successfully." << endl;
    }
    catch (exception &e)
    {
        cout << "Undo failed: " << e.what() << endl;
    }
}

void viewAllSites() { tree.traverseInorder(); }
AVLTree &getTree() { return tree; }

void showActionLog() // Display the action log
{
    try
    {
        ifstream file("Actions.txt");
        if (!file.is_open())
        {
            cout << "No actions logged.\n";
            return;
        }

        string line;
        while (getline(file, line))
            cout << line << endl;

        file.close();
    }
    catch (exception &e)
    {
        cout << "Error reading log file: " << e.what() << endl;
    }
}
```

```
    void exportAllSites() // Export all sites and usernames to file
    {
        logger.saveAllSitesAndUsernames(&tree);
        cout << "All sites and usernames exported to AllSites.txt\n";
    }
};

int main()
{
    AuthSystem auth; // Initialize authentication system
    auth.LoadMasterPassword();
    if (auth.getMasterHash().empty()) // Set master password if not set
        auth.setMasterPassword();
    if (!auth.authenticate()) // Authenticate user, exit if failed
        return 0;

    FileManager fm; // Initialize file manager and password vault
    PasswordVault vault(fm);
    fm.loadAll(&vault.getTree());

    while (true) // Main menu loop
    {
        cout << "\nMenu:\n1. Add Entry\n2. Delete Entry\n3. View Entry\n4. Undo"
             << "\n5. View All Sites\n6. View Action Log\n7. Save and Exit\nEnter Choice:
";
        string choiceStr;
        getline(cin, choiceStr);
        if (choiceStr.empty())
        {
            cout << "Choice cannot be empty. Please enter a valid choice.\n";
            continue;
        }
        stringstream ss(choiceStr);
        int choice;
        if (!(ss >> choice))
        {
            cout << "Invalid input. Enter a number.\n";
            continue;
        }
        if (choice < 1 || choice > 7)
        {
            cout << "Choice must be between 1 and 7.\n";
            continue;
        }

        switch (choice) // Process user choice
        {
        case 1:
            vault.addEntry();
            break;
        case 2:
```

```
            vault.deleteEntry();
            break;
        case 3:
            vault.viewEntry();
            break;
        case 4:
            vault.undoAction();
            break;
        case 5:
            vault.viewAllSites();
            break;
        case 6:
            vault.showActionLog();
            break;
        case 7:
            fm.saveAll(&vault.getTree());
            return 0;
        default:
            cout << "Invalid choice.\n";
            break;
        }
    }
    return 0;
}
```

12. Output

```
Enter Master Password: admin@project

Menu:
1. Add Entry
2. Delete Entry
3. View Entry
4. Undo
5. View All Sites
6. View Action Log
7. Save and Exit
Enter Choice: 1
Enter Site: www.google.com
Enter Username: user_123
Enter Password: passWord@123
Entry added.
```

```
Menu:
1. Add Entry
2. Delete Entry
3. View Entry
4. Undo
5. View All Sites
6. View Action Log
7. Save and Exit
Enter Choice: 3
Search by (1) Site or (2) Username:
Invalid choice. Please enter 1 or 2.
Search by (1) Site or (2) Username: 1
Enter Site: www.google.com
Site: www.google.com
Username: user_123
Password: ************
Reveal password? (y/n): n
```

```
Menu:
1. Add Entry
2. Delete Entry
3. View Entry
4. Undo
5. View All Sites
6. View Action Log
7. Save and Exit
Enter Choice: 5
Site: a.com | Username: user_123
Site: chrome.com | Username: user_chrome
Site: hotmail.com | Username: mailUser
Site: upwork.com | Username: FreeUser
Site: www.google.com | Username: user_123
Site: yahoo.com | Username: user@yahoo
```

```
Menu:
1. Add Entry
2. Delete Entry
3. View Entry
4. Undo
5. View All Sites
6. View Action Log
7. Save and Exit
Enter Choice: 6

[2025-12-18 22:03:23] Added site: yahoo.com
[2025-12-18 22:05:27] Added site: yahoo.com
[2025-12-18 22:43:09] Deleted site: twitter
[2025-12-18 22:44:13] Added site: hotmail.com
[2025-12-18 23:45:06] Added site: upwork.com
[2025-12-25 15:53:13] Added site: chroome.com
[2025-12-25 16:05:04] Added site: google.com
[2025-12-25 16:07:40] Undo add site: google.com
[2025-12-25 16:11:58] Added site: divine.com
[2025-12-25 16:12:06] Undo add site: divine.com
[2025-12-25 16:15:54] Added site: w3school.com
[2025-12-25 16:15:58] Undo add site: w3school.com
[2025-12-25 16:19:18] Added site: s3school.com
[2025-12-25 16:24:45] Deleted site: s3school.com
[2025-12-25 16:46:25] Added site: s3school.com
[2025-12-25 17:03:23] Added site: s3school.com
[2025-12-25 17:04:57] Deleted site: s3school.com
[2025-12-25 17:17:47] Deleted site: instagram
[2025-12-25 17:21:10] Deleted site: yahoo.com
[2025-12-25 17:21:28] Undo delete site: yahoo.com
[2025-12-25 17:27:26] Deleted site: yahoo.com
[2025-12-25 17:27:34] Undo delete site: yahoo.com
```

```
Menu:
1. Add Entry
2. Delete Entry
3. View Entry
4. Undo
5. View All Sites
6. View Action Log
7. Save and Exit
Enter Choice: 4
Undoing addition of site: www.google.com
Undo operation completed successfully.

Menu:
1. Add Entry
2. Delete Entry
3. View Entry
4. Undo
5. View All Sites
6. View Action Log
7. Save and Exit
Enter Choice: 2
Delete by (1) Site or (2) Username: 1
Enter Site: youtube.com
Nothing to delete.
```

```
Menu:
1. Add Entry
2. Delete Entry
3. View Entry
4. Undo
5. View All Sites
6. View Action Log
7. Save and Exit
Enter Choice: 7
```

THE END